From data to figure

Here I present my annotations to the original code Ben shared with me (`moments_script_STANCE.py`).

Note. Tips for readability

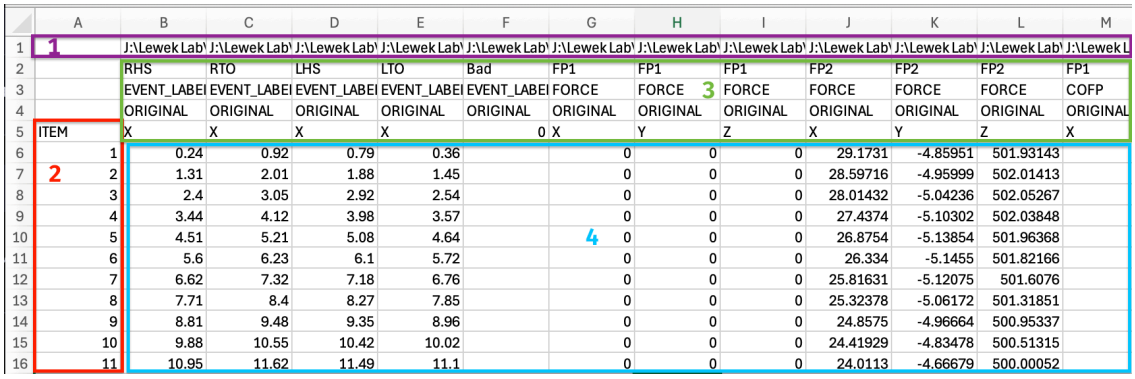
PEP 8, the official *Python Style Guide*, recommends:

- **Limit all lines to a maximum of 79 characters** for code.
- **Limit docstrings and comments to 72 characters**.

These days, people use wider monitors, so teams relax this rule to **88 characters**. So going over *79 characters* is not a dealbreaker *per se*, but it's also a good idea to limit your codes and comments rather *short*.

1. Reading a file with complex structure

The code reads an `.xls` file (`Shefaali_Walk1_Baseline_Data.xls`) that appears to be an output of a motion capture system.



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	1	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek Lab\	J:\Lewek L
2		RHS	RTO	LHS	LTO	Bad	FP1	FP1	FP1	FP2	FP2	FP2	FP1
3		EVENT_LABEI	EVENT_LABEI	EVENT_LABEI	EVENT_LABEI	EVENT_LABEI	FORCE	FORCE 3	FORCE	FORCE	FORCE	FORCE	COFP
4		ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL
5	ITEM	X	X	X	X	0	X	Y	Z	X	Y	Z	X
6	1	0.24	0.92	0.79	0.36		0	0	0	29.1731	-4.85951	501.93143	
7	2 2	1.31	2.01	1.88	1.45		0	0	0	28.59716	-4.95999	502.01413	
8	3	2.4	3.05	2.92	2.54		0	0	0	28.01432	-5.04236	502.05267	
9	4	3.44	4.12	3.98	3.57		0	0	0	27.4374	-5.10302	502.03848	
10	5	4.51	5.21	5.08	4.64		4 0	0	0	26.8754	-5.13854	501.96368	
11	6	5.6	6.23	6.1	5.72		0	0	0	26.334	-5.1455	501.82166	
12	7	6.62	7.32	7.18	6.76		0	0	0	25.81631	-5.12075	501.6076	
13	8	7.71	8.4	8.27	7.85		0	0	0	25.32378	-5.06172	501.31851	
14	9	8.81	9.48	9.35	8.96		0	0	0	24.8575	-4.96664	500.95337	
15	10	9.88	10.55	10.42	10.02		0	0	0	24.41929	-4.83478	500.51315	
16	11	10.95	11.62	11.49	11.1		0	0	0	24.0113	-4.66679	500.00052	

I usually open a file before writing codes. This is to figure out how to read a file. For example, the first row (1) repeatedly stores the path where a `.c3d` file is saved. I can skip it using `skiprows`.

In [1]:
```
# You use read_excel() to read .xls files
import pandas as pd
data = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls', skiprows=1)

# I usually check the output immediately.
data.head()
```

Out[1]:

	Unnamed: 0	RHS	RTO	LHS	LTO	
0	NaN	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL	EVENT
1	NaN	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	OF
2	ITEM	X	X	X	X	
3	1	0.24	0.92	0.79	0.36	
4	2	1.31	2.01	1.88	1.45	

5 rows × 38 columns

Good. It's read fine. The first column is 'Unnamed: 0', because in the `.xls` file, it's a blank cell. In fact, the entire column (2) is just indices of the values. This we can skip as well.

We also see that on the right, column headers are **FP1**, **FP1.1**, and **FP1.2**. In the original file, these three columns are all **FP1**, but pandas need to have *unique* column headers, so numbers are attached to identical headers.

However, if you look closely, the first four rows (3) in each column would together make a unique column header. Can we make use of that?

In [2]:
```
# When you give what rows to use as headers,
# you can provide a list of row indices.
# Here, we want the first four rows (0, 1, 2, 3) to be used
# as headers.
# Remember, this will prepare multiIndex (discussed in Week 3)
data = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls', skiprows=1,
                     header=[0, 1, 2, 3])

# You can then drop the first column, which is just indices.
# Using `data.columns[0]` gets the name of the first column.
data = data.drop(columns=data.columns[0])
data.head()
```

Out[2]:

	RHS EVENT_LABEL ORIGINAL X	RTO EVENT_LABEL ORIGINAL X	LHS EVENT_LABEL ORIGINAL X	LTO EVENT_LABEL ORIGINAL X	Bad EVENT_LABEL ORIGINAL 0
0	0.24	0.92	0.79	0.36	NaN
1	1.31	2.01	1.88	1.45	NaN
2	2.40	3.05	2.92	2.54	NaN
3	3.44	4.12	3.98	3.57	NaN
4	4.51	5.21	5.08	4.64	NaN

5 rows × 37 columns

In [3]:
```
# See that it's a MultiIndex now.
data.columns[0:5]
```

Out[3]:
```
MultiIndex([('RHS', 'EVENT_LABEL', 'ORIGINAL', 'X'),
            ('RTO', 'EVENT_LABEL', 'ORIGINAL', 'X'),
            ('LHS', 'EVENT_LABEL', 'ORIGINAL', 'X'),
            ('LTO', 'EVENT_LABEL', 'ORIGINAL', 'X'),
            ('Bad', 'EVENT_LABEL', 'ORIGINAL',   0)],
           )
```

Comparison with the original method I

Ben's original code (lines 1-83) proceeds in this manner:

1. read the file repeatedly to save column headers at different levels to different variables.
2. read the excel file skipping 5 rows and assign headers of choice

In [4]:
```
# Adapted from the original code for conciseness.
column_names = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                             header=None, nrows=2)
footcolumn_names = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                                 header=None, nrows=5)
true_headers = column_names.iloc[1]
foot_labels = footcolumn_names.iloc[3]
df = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                   header=None, skiprows=5)
df.columns = true_headers
df.head()
```

Out[4]:

	1	NaN	RHS	RTO	LHS	LTO	Bad	FP1	FP1	FP1	FP2	...	Left Knee Moment	L M
0	1	0.24	0.92	0.79	0.36	NaN	0.0	0.0	0.0	29.17310	...	NaN		
1	2	1.31	2.01	1.88	1.45	NaN	0.0	0.0	0.0	28.59716	...	NaN		
2	3	2.40	3.05	2.92	2.54	NaN	0.0	0.0	0.0	28.01432	...	-0.01031	-(	
3	4	3.44	4.12	3.98	3.57	NaN	0.0	0.0	0.0	27.43740	...	-0.01316	-(	
4	5	4.51	5.21	5.08	4.64	NaN	0.0	0.0	0.0	26.87540	...	-0.03113	-(	

5 rows × 38 columns

This can be one way, but we see that the first column's name is `NaN`, and there are duplicate names (`FP1`). This will be tricky, because there's no other information provided other than the names, so you don't know which column corresponds to which axis.

In [5]:
```
# There are six columns named 'FP1'.
# The first three are "FORCE, x, y, and z axis measures".
# The next three are "COFP, x, y, and z axis measures".
# Right now we can't tell which is which
# unless we have domain knowledge.
df['FP1'].head()
```

Out[5]:

	1	FP1	FP1	FP1	FP1	FP1	FP1
0	0.0	0.0	0.0	NaN	NaN	NaN	
1	0.0	0.0	0.0	NaN	NaN	NaN	
2	0.0	0.0	0.0	NaN	NaN	NaN	
3	0.0	0.0	0.0	NaN	NaN	NaN	
4	0.0	0.0	0.0	NaN	NaN	NaN	

In [6]:
```
# In comparison, with MultiIndex, you can be specific.
data[('FP1', 'FORCE', 'ORIGINAL', 'X')].head()
```

Out[6]:
```
0    0.0
1    0.0
2    0.0
3    0.0
4    0.0
Name: (FP1, FORCE, ORIGINAL, X), dtype: float64
```

In [7]:
```
# You can also get all FORCE data for FP1.
data[('FP1', 'FORCE')].head()
```

```
/var/folders/b4/_bb0snyn7nn6f7d4sk8ry5jr0000gn/T/ipykernel_41250/305361
139.py:2: PerformanceWarning: indexing past lexsort depth may impact pe
rformance.
  data[('FP1', 'FORCE')].head()
```

Out[7]:

	ORIGINAL		
	X	Y	Z
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

2. Manipulate original columns to make new columns

Ben's original code (lines 96-125) then goes manipulate columns to create new columns: `Left_CGVel_Avg` and `Right_CGVel_Avg`.

Using my `data`, we can do it this way:

In [8]:
```
data[('CGVel', 'KINETIC_KINEMATIC', 'LFT', 'Avg')] =\
      data.loc[:, pd.IndexSlice['CGVel', :, 'LFT', :]].mean(axis=1)
data[('CGVel', 'KINETIC_KINEMATIC', 'RFT', 'Avg')] =\
      data.loc[:, pd.IndexSlice['CGVel', :, 'RFT', :]].mean(axis=1)

# Show the resulting columns
data.loc[:, pd.IndexSlice['CGVel', :, :, 'Avg']].head()
```

Out[8]:

	CGVel	
	KINETIC_KINEMATIC	
	LFT	RFT
	Avg	Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

In [9]:
```
# pd.IndexSlice helps select specific levels of MultiIndex.
# The line below will select all columns
# where the first level is 'CGVel',
# the third level is 'LFT', and the other levels can be anything.
# ('Avg' is included because we ran the line above to create it.)
data.loc[:, pd.IndexSlice['CGVel', :, 'LFT', :]].head()
```

Out[9]:

	CGVel	
	KINETIC_KINEMATIC	
	LFT	
	Y	Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	-1.21126
3	-1.22225	-1.22225
4	-1.22255	-1.22255

In [10]:
```
# Of course, if you don't know what the second level header is,
# you can always check the columns of the DataFrame.
# Use `.get_level_values()` to get specific level values.
# For example, to get all first level headers that are 'CGVel':
cgvel_multi_index = data.columns[
    data.columns.get_level_values(0) == 'CGVel']

# Cehck what `cgvel_multi_index` looks like
cgvel_multi_index
```

Out[10]:
```
MultiIndex([('CGVel', 'KINETIC_KINEMATIC', 'LFT',   'Y'),
            ('CGVel', 'KINETIC_KINEMATIC', 'RFT',   'Y'),
            ('CGVel', 'KINETIC_KINEMATIC', 'LFT', 'Avg'),
            ('CGVel', 'KINETIC_KINEMATIC', 'RFT', 'Avg')],
           )
```

In [11]:
```
# We need to get 'KINETIC_KINEMATIC',
# which is the second level header.
# Note that the value for get_level_values() is 1
# because it's zero-indexed.
# `.unique()` gets unique values only.
# Then indexing this output using `[0]`
# gets the first (and only) value.
second_level_headers = cgvel_multi_index.get_level_values(1).unique()
kinematic_header = second_level_headers[0]

# You can then use it in the code above:
# (Don't run lines below again; they are just for illustration.
```

```
# data[('CGVel', kinematic_header, 'LFT', 'Avg')] =\
#       (data.loc[:, pd.IndexSlice['CGVel', kinematic_header, 'LFT', :
#       .mean(axis=1))
# data[('CGVel', kinematic_header, 'RFT', 'Avg')] =\
#       (data.loc[:, pd.IndexSlice['CGVel', kinematic_header, 'RFT', :
#       .mean(axis=1))
```

Comparison with the original method II

This is how the original code works:

1. prepare empty lists, and append all columns with the header: `CGVel` if the column's `foot_label` either matches `"LFT"` or `"RFT"`.
2. calculate columnwise means each for `"LFT"` and `"RFT"`.

In [12]:
```
# This is the current DataFrame loaded.
# We see that the two columns are indistinguishable
# because they have the same name 'CGVel'.
df.loc[:, df.columns == 'CGVel'].head()
```

Out[12]:

1	CGVel	CGVel
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

In [13]:
```
# Ben's code (lines 96-125) goes like this:
left_cgvel_idxs = []
right_cgvel_idxs = []

for i, col in enumerate(df.columns):
    if "CGVEL" in str(col).upper():
        label = str(foot_labels.iloc[i]).strip().upper()
        if label == "LFT":
            left_cgvel_idxs.append(i)
        elif label == "RFT":
            right_cgvel_idxs.append(i)

if left_cgvel_idxs:
    df['Left_CGVel_Avg'] = df.iloc[:, left_cgvel_idxs].mean(axis=1)
    left_cgvel_col = "Left_CGVel_Avg"
else:
    left_cgvel_col = None

if right_cgvel_idxs:
```

```
    df['Right_CGVel_Avg'] = df.iloc[:, right_cgvel_idxs].mean(axis=1)
    right_cgvel_col = "Right_CGVel_Avg"
else:
    right_cgvel_col = None

# Check the output
df[['Left_CGVel_Avg', 'Right_CGVel_Avg']].head()
```

Out[13]:

1	Left_CGVel_Avg	Right_CGVel_Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

This is also a valid method - you just need to do more mental juggling!

3. Helper functions

Functions are prepared in the original code:

1. `extract_peak_per_stride(joint_label, direction='max')`:

   extract peak moments of a joint (ex. Right Knee) per stride

2. `to_frames(col_name)`:

   convert gait events from seconds to frame indices

3. `average_middle_values(arr)`:

   average middle values (this is not extensively used)

This section you can skip, if you don't find too relevant.

What can be improved?

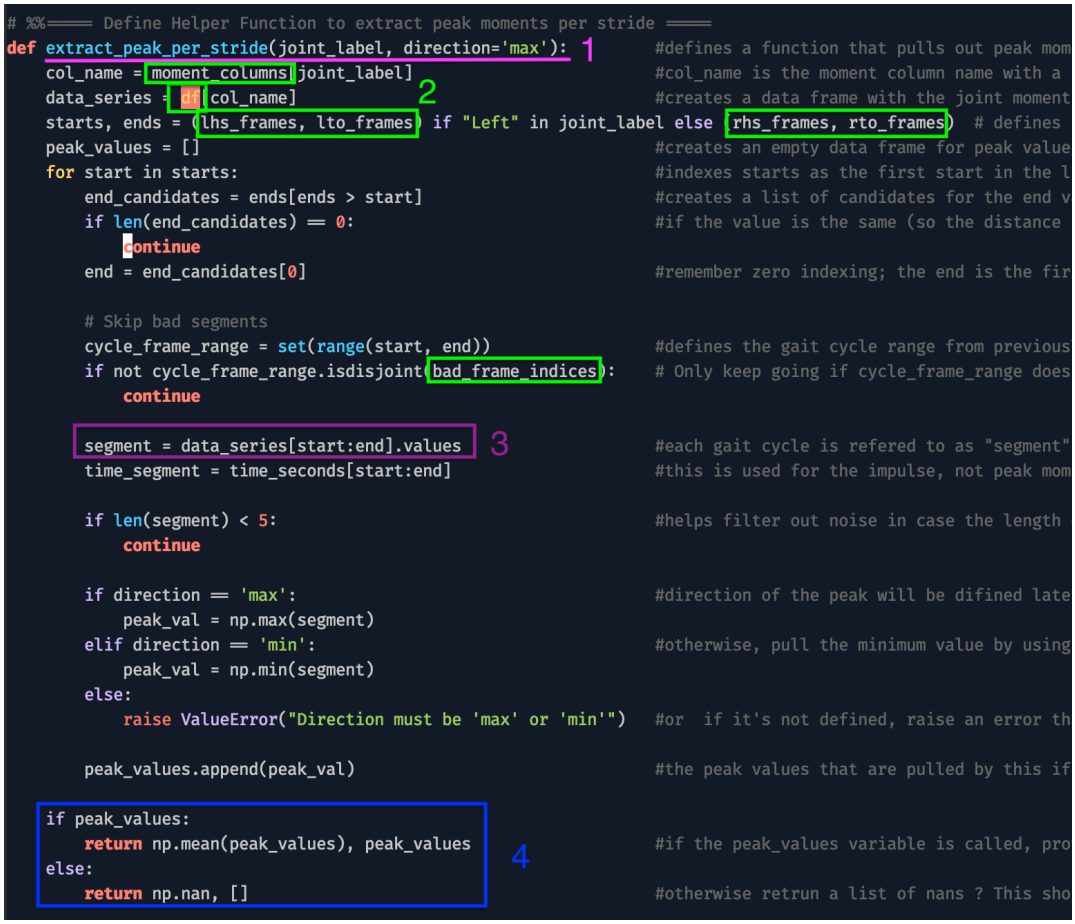
1. **Docstring**: I suggest using docstring. Also, keep in mind of the style recommendation of **PEP 8**.

2. **local variables** vs. **global variables**: `extract_peak_per_stride` uses variables defined **outside** (ex. `moment_columns` or `df`). If you accidentally erase these variables or change the names of them, this function

would not operate until you reflect such changes. This is a risky behavior. I recommend minimizing the use of variables defined outside a function.

3. **foolproof indexing**: use `.iloc` if you use indices.

4. **consistent output type**: if you return `numpy.ndarray` for one, do the same for another.



1. Docstring

This is an example of Numpy style docstring.

In [ ]:
```
"""
Extract peak moments per stride for a given joint.

Parameters
----------
df: pd.DataFrame
    The DataFrame containing gait data.
joint_label: str
    The label of the joint to extract data for.
    (ex. 'Right Ankle', 'Left Knee', etc.)
direction: str
    'max' for maximum peaks, 'min' for minimum peaks.
```

```
sampling_rate: float
    The sampling rate of the data in Hz; default is 100.0 Hz.

Returns
-------
np.array
    mean of peak moments per stride
    and all peak moments per stride.
"""
```

2. Local variables & Parameters

1. More parameters to the function (ex. `df`, `sampling_rate`)
2. Give type hints (ex. `df: pd.DataFrame`).
3. Parameters with default values will be placed after those without default values.

   ```
   def extract_peak_per_stride(df: pd.DataFrame, joint_label: str,
                               direction: str = 'max',
                               sampling_rate: float = 100.0)
   ```
4. Other variables are defined inside the function.
   - `moment_columns`
   - `*_frames` (ex. `lhs_frames`)
   - `bad_frame_indices`

In [14]:
```
# This is how moment_columns is defined in the original code.
#   moment_columns = {
#       "Right Ankle": "Right_Ankle_Moment",
#       "Left Ankle": "Left_Ankle_Moment",
#       "Right Knee": "Right_Knee_Moment",
#       "Left Knee": "Left_Knee_Moment",
#       "Right Hip": "Right_Hip_Moment",
#       "Left Hip": "Left_Hip_Moment"
#   }

# They are all headers with "Moment"
# So we can extract a header with "Moment"
# and the joint label (ex. "Right Ankle")
joint_label = 'Right Ankle'
moment_columns = [x for x in data.columns if "Moment" in x[0] and join
moment_columns
```

Out[14]:
```
[('Right Ankle Moment', 'LINK_MODEL_BASED', 'ORIGINAL', 'X')]
```

In [15]:
```
# lhs_frames, rhs_frames, lto_frames, rto_frames
# are defined in the original code:
```

```
#   rhs_frames = to_frames(event_columns["RHS"])

# `to_frames` is a function with the following definition
# in the original code:
#   def to_frames(event_times, sampling_rate=100.0):
#       return ((df[col_name].dropna().astype(float) * sampling_rate)
#               .astype(int).values)

# The original function again uses `df` defined outside the function.
# `.astype(float)` ensures the data type is float
# before multiplication, but not necessary if values are multiplied
# by 100.0 (float).
# `.values` converts the Series to a numpy array.
# It's recommended to use `.to_numpy()`.

# We would define the function like this:
# 1) `df` is passed as a parameter.
# 2) `.loc` is used for indexing because we are using MultiIndex.
#    col_name is expected to be a pd.IndexSlice object.
# 3) `.ravel()` is used to convert 2D array to 1D array.
# 4) `.tolist()` is used to convert to a plain list.
def to_frames(df: pd.DataFrame, col_name: tuple | slice,
              sampling_rate:float=100.0):
    """Convert gait event times from seconds to frame indices."""
    vals = df.loc[:, col_name].dropna().to_numpy().ravel()
    return (vals * sampling_rate).astype(int).tolist()
```

In [ ]:

```
# `bad_frame_indices` is defined in the original code as:
#   bad_frame_indices = set()
#   if event_columns["Bad"] in df.columns:
#       bad_col = df[event_columns["Bad"]].astype(str).fillna("")
#       for val in bad_col:
#           try:
#               frame = int(round(float(val) * sampling_rate))
#               bad_frame_indices.add(frame)
#           except ValueError:
#               continue

# Here, `event_columns` is defined as a dictionary elsewhere.
# (ex. {'RHS': 'RHS', 'LHS': 'LHS', ...})

# Not sure what's the data type of that column,
# but if converting to `float` is possible for some values,
# I assume the column itself is numeric to begin with.
# Thus we can skip the `.astype(str)` step.
# Also, `frame` can be calculated using `to_frames`.
# For example, if 'Bad' label is similar to 'RHS' label (ex. 8.81),
# then `float(val) * sampling_rate` -> 881.0
# In other words, as long as labels are printed to
# the second decimal point, and the sampling_rate is fixed at 100.0,
# `round` is redundant.
```

```
# This is how I would rewrite it:
# try:
#   bad_frame_indices = set(
#       to_frames(df, pd.IndexSlice['Bad', :, :, :], sampling_rate))
# except:
#   bad_frame_indices = set()

# This way, even if `try` section fails,
# we will have `bad_frame_indices`, which is an empty set.
```

3. `.iloc` for indexing

Minor edit, but just to make sure. Also, `end+1` will make sure `end` index is included.

`segment = data_series[start:end].values` -> `segment = data_series.iloc[start:end+1]`

4. Consistent return data types

If one of the two returned item is `np.ndarray`, consider making the other one to be the same.

Putting everything together...

In [16]:
```
# Here's how I would prepare:
# `df` as a parameter, and docstring added.
import numpy as np


def to_frames(df: pd.DataFrame, col_name: tuple | slice,
              sampling_rate:float=100.0):
    """
    Convert gait event times from seconds to frame indices.

    Parameters
    ----------
    df: pd.DataFrame
        The DataFrame containing gait data.
    col_name: tuple | slice
        Column header whose values will be converted (ex. 'RHS')
    sampling_rate: float
        Sampling rate of the measurement device; default is 100.0

    Returns
    -------
    list of frame indices
    """
    vals = df.loc[:, col_name].dropna().to_numpy().ravel()
```

```
    return (vals * sampling_rate).astype(int).tolist()


def extract_peak_per_stride(df: pd.DataFrame,
                            joint_label: str,
                            direction: str = 'max',
                            sampling_rate: float = 100.0):
    """
    Extract peak moments per stride for a given joint.

    Parameters
    ----------
    df: pd.DataFrame
        The DataFrame containing gait data.
    joint_label: str
        The label of the joint to extract data for.
        (ex. 'Right Ankle', 'Left Knee', etc.)
    direction: str
        'max' for maximum peaks, 'min' for minimum peaks.
    sampling_rate: float
        The sampling rate of the data in Hz; default is 100.0 Hz.

    Returns
    -------
    mean of peak moments per stride and all peak moments per stride as
    """

    # Make sure user input is valid
    if direction not in ['max', 'min']:
        raise ValueError("Direction must be either 'max' or 'min'.")

    # Frame indices
    # HS: Heel Strike; TO: Toe Off
    # For example, 'RHS' is a series
    # (0.24, 1.31, 2.4, ..., 28.9, 29.96)
    # In terms of frame indices (`rhs_frames`),
    # it will be (24, 131, 240, ..., 2890, 2996)
    # given a sampling rate of 100 Hz.
    rhs_frames = to_frames(df=df,
                           col_name=pd.IndexSlice['RHS', :, :, :],
                           sampling_rate=sampling_rate)
    # If you provide parameters in the right order,
    # you don't need to specify which value is for which parameter.
    rto_frames = to_frames(df,
                           pd.IndexSlice['RTO', :, :, :],
                           sampling_rate)
    lhs_frames = to_frames(df,
                           pd.IndexSlice['LHS', :, :, :],
                           sampling_rate)
    lto_frames = to_frames(df,
                           pd.IndexSlice['LTO', :, :, :],
                           sampling_rate)
```

```
    col_name = [x for x in df.columns
                if "Moment" in x[0] and joint_label in x[0]]
    data_series = df[col_name]
    starts, ends = (lhs_frames, lto_frames) if "Left" in \
        joint_label else (rhs_frames, rto_frames)
    # Ok, HS can follow 'TO'
    if starts[0] > ends[0]:
        ends = ends[1:]

    try:
        bad_frame_indices = set(
            to_frames(df, pd.IndexSlice['Bad', :, :, :], sampling_rate
    except:
        bad_frame_indices = set()

    peak_values = []
    # Using `zip()` can skip finding `end` by
    # creating another variable: `end_candidates`.
    # `zip()` stops at the shortest input,
    # so no IndexError will occur.
    for start, end in zip(starts, ends):
        # Skip short segments (less than 5 frames).
        # This replaces `if len(segment) < 5: continue`
        if end - start < 5:
            continue
        # If 'bad' frame exists in this cycle, skip it.
        cycle_frame_range = set(range(start, end))
        # Original logic is double negative (`not isdisjoint`)
        # Alternatively, use `&` operator or `.intersection` method
        # to make a set whose elements are both present in
        # `cycle_frame_range` and `bad_frame_indices`
        # and if that set has length > 0, `continue`
        if (cycle_frame_range & bad_frame_indices):
            continue

        segment = data_series.iloc[start:end]
        peak_val = segment.max() if direction == 'max' \
            else segment.min()
        peak_values.append(peak_val)

    # Originally, a numpy array and
    # a list (peak_values) were returned.
    # Be more consistent by returning a numpy array for both.
    # Along with np.nan, return an empty numpy array,
    # not an empty list.
    if peak_values:
        return np.mean(peak_values), np.array(peak_values)
    else:
        return np.nan, np.array([])
```

Let's check if the function works as intended. Let's check if it can extract peak

moment per stride using the **Right Knee** moments. We will be finding the **maximum** of segments.

Here are the times (second) of the right heel strikes (RHS) and right toe offs (RTO).

```
+======+======+
| RHS  | RTO  |
+------+------+
| 0.24 | 0.92 |
+------+------+
| 1.31 | 2.02 |
+------+------+
| ...  | ...  |
+------+------+
| 28.9 | 29.58|
+------+------+
| 29.96| NA   | <- This will not be included
+------+------+
```

The first segment will be from 0.24s - 0.92s, or 24th frame to 92th frame.

In [17]:
```
# Test: Right Knee Moment
pk_mean, pks = extract_peak_per_stride(data, "Right Knee", 'max')
# This is the max of the first segment
pks[0]
```

Out[17]:
```
array([0.56576])
```

In [18]:
```
# Check the segment:
segment = data.loc[24:93,
                   ('Right Knee Moment',
                    'LINK_MODEL_BASED', 'ORIGINAL', 'X')]
# Maximum of the segment: is it the same as `pks[0]`?
max(segment)
```

Out[18]:
```
0.56576
```

In [19]:
```
# Test 2: Left Hip Moment
pk_mean_lhip, pks_lhip = extract_peak_per_stride(
    data, "Left Hip", 'min')
# Let's try the second segment (1.88 ~ 2.54s)
segment_lhip = data.loc[131:255,
                        ('Left Hip Moment',
                         'LINK_MODEL_BASED', 'ORIGINAL', 'X')]
# Is it `True`?
pks_lhip[1] == min(segment_lhip)
```

Out[19]:
```
array([ True])
```

4. More data manipulation

This section is really not relevant to most...

Stance velocities (lines 216-251)

In [20]:
```python
# lines 216-251
# ----- previous info needed for demo purpose -----
sampling_rate = 100.0
rhs_frames = to_frames(df=data,
                       col_name=pd.IndexSlice['RHS', :, :, :],
                       sampling_rate=sampling_rate)
rto_frames = to_frames(data,
                       pd.IndexSlice['RTO', :, :, :],
                       sampling_rate)
lhs_frames = to_frames(data,
                       pd.IndexSlice['LHS', :, :, :],
                       sampling_rate)
lto_frames = to_frames(data,
                       pd.IndexSlice['LTO', :, :, :],
                       sampling_rate)

# ----- cleaning codes -----
# Here we can prepare a small function that returns a list of tuples:
# (start, end)
# This can also be used inside `extract_peak_per_stride`

def return_windows(hs_frames: list, to_frames: list) -> list:
    """
    From a HS (heel strike) and a TO (toe off) frame series,
    make a list of tuples of gait start and end frame.

    Parameters
    ----------
    hs_frames: list
        List of heel strike frames; output of `to_frames()`
    to_frames: list
        List of toe off frames; output of `to_frames()`

    Returns
    -------
    list of tuples
    """
    # If the first 'start' is later than the potential 'first' end,
    # take the next end to pair with this first start.
    if hs_frames[0] > to_frames[0]:
        return list(zip(hs_frames, to_frames[1:]))
    return list(zip(hs_frames, to_frames))
```

```
# For example,
left_frames = return_windows(lhs_frames, lto_frames)
left_frames[0:5]
```

Out[20]: [(79, 145), (188, 254), (292, 357), (398, 463), (508, 572)]

In [21]:
```
# Then define another wrapper function
def get_stance_velocities(df: pd.DataFrame,
                          side: str = "Left",
                          sampling_rate: float = 100) -> list:
    """
    Parameters
    ----------
    df: pd.DataFrame
        Gait data with CGVel average columns populated.
    side: str
        Which leg data to analyze; default is "Left"
    sampling_rate: float
        The sampling rate of the data in Hz; default is 100.0 Hz.

    Returns
    -------
    middle_avgs: list
        List of stance velocities
    """
    middle_avgs = list()

    if side == "Left":
        gait_frames = return_windows(
            to_frames(df, pd.IndexSlice['LHS', :, :, :],
                      sampling_rate),
            to_frames(df, pd.IndexSlice['LTO', :, :, :],
                      sampling_rate))
        col_label = "LFT"
    else:
        gait_frames = return_windows(
            to_frames(df, pd.IndexSlice['RHS', :, :, :],
                      sampling_rate),
            to_frames(df, pd.IndexSlice['RTO', :, :, :],
                      sampling_rate))
        col_label = "RFT"

    try:
        bad_frame_indices = set(
            to_frames(data, pd.IndexSlice['Bad', :, :, :],
                      sampling_rate))
    except:
        bad_frame_indices = set()

    for start, end in gait_frames:
        cycle_range = set(range(start, end))
        if cycle_range.isdisjoint(bad_frame_indices):
```

```
            avg_slice = pd.IndexSlice["CGVel", :, col_label, "Avg"]
            cgvel_segment = df.loc[start:end, avg_slice].to_numpy()
            backward_values = cgvel_segment[cgvel_segment < 0]
            if len(backward_values) >= 5:
                n = len(backward_values)
                start_idx = int(0.1*n)
                end_idx = int(0.6*n)
                middle_avgs.append(
                    backward_values[start_idx:end_idx].mean())

    return middle_avgs
```

In [ ]:
```
# How to use it:
left_stance_velocities = get_stance_velocities(data, "Left")
right_stance_velocities = get_stance_velocities(data, "Right")
```

Ensemble averaging (lines 207-393)

In [23]:
```
# Another helper function:
def make_segment(df: pd.DataFrame,
                 joint_label: str,
                 start: int,
                 end: int):
    col_name = [x for x in df.columns
                if "Moment" in x[0] and joint_label in x[0]]
    data_series = df[col_name]

    return data_series.iloc[start:end].to_numpy().ravel()
```

In [24]:
```
from scipy.interpolate import interp1d

def interpolate_segments(df: pd.DataFrame,
                         joint_label: str,
                         sampling_rate: float = 100.0):

    time_seconds = np.arange(len(df)) / sampling_rate
    n_points = 101

    bad_segment_count = 0
    all_cycles = list()
    impulses = list()
    impulses_ext = list()
    impulses_flex = list()

    if "Left" in joint_label:
        gait_frames = return_windows(
            to_frames(df,
                      pd.IndexSlice['LHS', :, :, :],
                      sampling_rate),
            to_frames(df,
                      pd.IndexSlice['LTO', :, :, :],
```

```
                    sampling_rate))
    else:
        gait_frames = return_windows(
            to_frames(df,
                      pd.IndexSlice['RHS', :, :, :],
                      sampling_rate),
            to_frames(df,
                      pd.IndexSlice['RTO', :, :, :],
                      sampling_rate))

    try:
        bad_frame_indices = set(
            to_frames(data,
                      pd.IndexSlice['Bad', :, :, :],
                      sampling_rate))
    except:
        bad_frame_indices = set()

    for start, end in gait_frames:
        cycle_frame_range = set(range(start, end))

        if (cycle_frame_range & bad_frame_indices):
            bad_segment_count += 1
            continue

        segment = make_segment(df, joint_label, start, end)
        time_segment = time_seconds[start:end]

        if len(segment) < 5:
            continue

        x_old = np.linspace(0, 1, len(segment))
        interpolator = interp1d(x_old, segment, kind='linear')
        normalized = interpolator(np.linspace(0, 1, n_points))
        all_cycles.append(normalized)

        if "Ankle" in joint_label:
            choose = segment < 0
            impulses.append(
                np.trapezoid(segment[choose],
                             time_segment[choose]) \
                    if np.any(choose) else np.nan
            )
        elif "Knee" in joint_label:
            choose = segment > 0
            impulses.append(
                np.trapezoid(segment[choose],
                             time_segment[choose]) \
                    if np.any(choose) else np.nan
            )
        elif "Hip" in joint_label:
            choose_ext = segment < 0
```

```
            choose_flex = segment > 0
            if np.any(choose_ext):
                impulses_ext.append(
                    np.trapezoid(segment[choose_ext],
                                 time_segment[choose_ext]))
            if np.any(choose_flex):
                impulses_flex.append(
                    np.trapezoid(segment[choose_flex],
                                 time_segment[choose_flex]))

    return {"bad_segment_counts": bad_segment_count,
            "all_cycles": np.asarray(all_cycles),
            "impulses": impulses,
            "impulses_ext": impulses_ext,
            "impulses_flex": impulses_flex}
```

5. Plotting using a for loop

In [25]:
```
import matplotlib.pyplot as plt

metadata = {
    "Subject": "Shefaali",
    "Session": "",
    "Condition": "",
    "Walk": "BaselineWalk",
    "GaitSpeed": "1.2"
}

# Here subplot locations (row, column) are paired with
# `joint_label`. This can be one good way to designate
# which variable will be plotted where.
subplot_layout = [
    ("Left Ankle", 0, 0), ("Right Ankle", 0, 1),
    ("Left Knee", 1, 0),  ("Right Knee", 1, 1),
    ("Left Hip", 2, 0),   ("Right Hip", 2, 1)
]

# `layout='constrained'` is useful when you have subplots
# and want to adjust space between them so that no overlap
# of axis labels happen.
fig, axes = plt.subplots(3, 2, figsize=(12, 10),
                         sharex=True, layout='constrained')
# Unpacking the three elements (ex. "Left Ankle", 0, 0)
# Here's one good use of using for loop to iterate over
# variables you want to plot in one figure.
for joint_label, row, col in subplot_layout:
    # Remember, `data` is the DataFrame we read.
    # `interpolate_segments` is the function defined in Section 4
    # It basically time normalizes data, so that moments at
    # different joint can be plotted using the same scale.
```

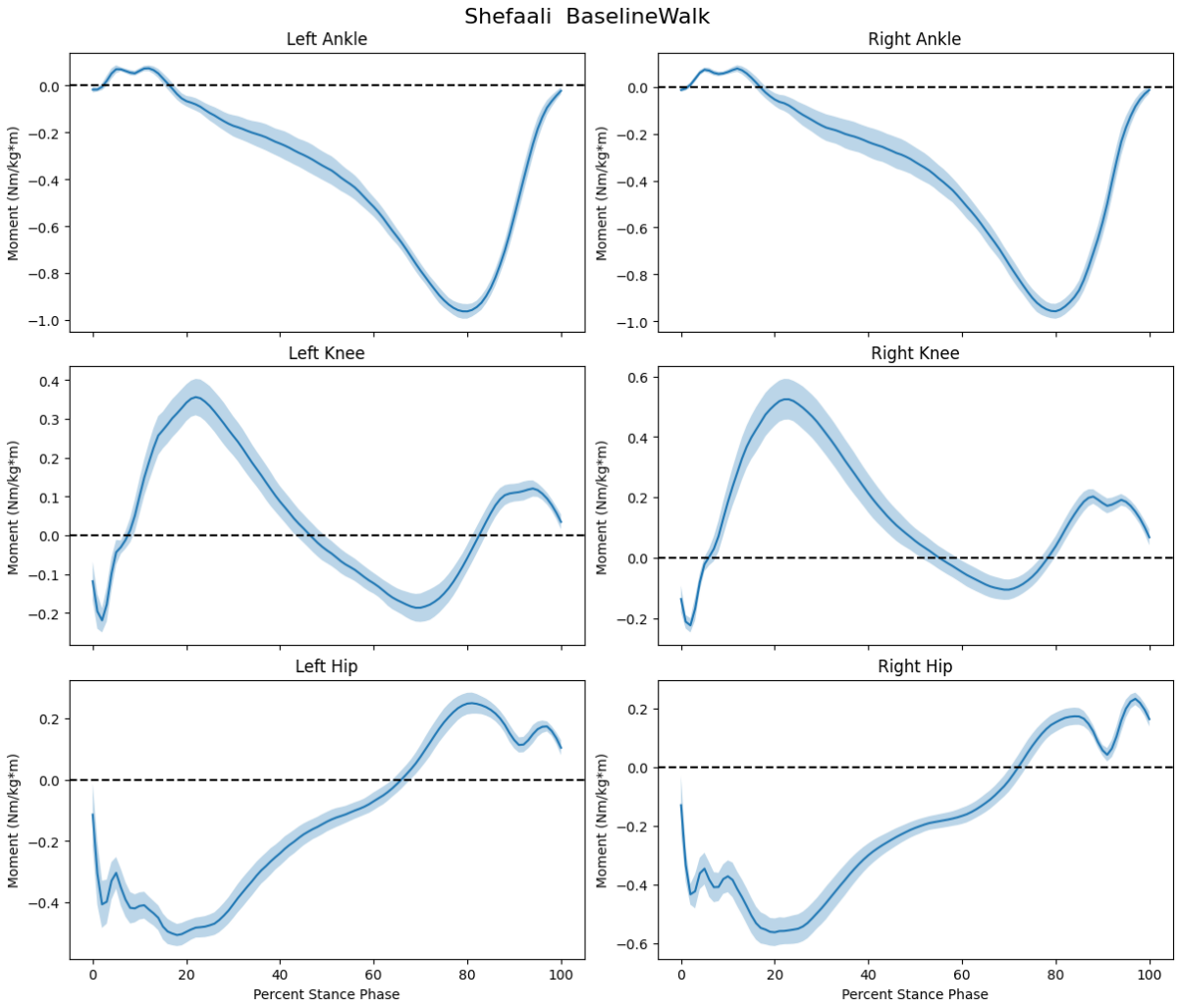
```
    intp_results = interpolate_segments(data, joint_label)
    ax = axes[row, col]

    all_cycles = intp_results['all_cycles']
    x_new = np.arange(0, 101)
    if len(all_cycles):
        mean_cycle = all_cycles.mean(axis=0)
        std_cycle = all_cycles.std(axis=0)
        ax.plot(x_new, mean_cycle, label='Mean')
        ax.fill_between(x_new, mean_cycle - std_cycle,
                        mean_cycle + std_cycle, alpha=0.3)

        ax.set_title(joint_label)
        ax.set_ylabel("Moment (Nm/kg*m)")
        ax.grid(False)
        ax.axhline(0, c='k', linestyle='--')

# As x-axis is shared (`sharex=True`),
# only set the last x-axis labels.
axes[2, 0].set_xlabel("Percent Stance Phase")
axes[2, 1].set_xlabel("Percent Stance Phase")
# Figure-level title.
fig.suptitle(f"{metadata['Subject']} "
             f"{metadata['Session']} "
             f"{metadata['Walk']} "
             f"{metadata['Condition']}",
             fontsize=16, y=1.02)
# Uncomment and run the line below to save this figure.
# fig.savefig("Some_figure.png", dpi=300, bbox_inches='tight')
```

Out[25]: `Text(0.5, 1.02, 'Shefaali  BaselineWalk ')`


Shefaali BaselineWalk
Left Ankle
Right Ankle
Left Knee
Right Knee
Left Hip
Right Hip
Moment (Nm/kg*m)
Percent Stance Phase